[CSE299(2) - Group 03] Weekly Report: AI Diff Checker

Name : Md Sanaul Islam Zihad
Student ID : 2311774042
Email : sanaul.zihad@northsouth.edu

I. Weekly Report 1

A. *Follow-up from last week*

No previous tasks assigned.

B. *Work narrative of the current week*

- **Tasks completed:**
  - At the beginning of this week, we decided to build our frontend using React since our project requires a modern and interactive user interface and it can easily communicate with our Python backend later on. After deciding this, we set up the React environment and started working with its basic structure.

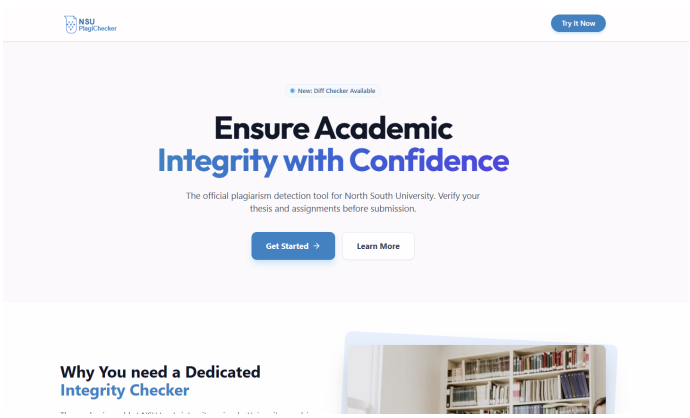

Fig. 1: Frontend Landing Page Design

  - After getting familiar with React, we designed a landing page for our plagiarism detection system. The landing page includes the project overview, system purpose, and navigation buttons so users can easily understand how the system works.
  - Next, we worked on designing a submission page where users can upload their documents for plagiarism checking. Currently, the "Try Now" button on the landing page takes the user to the checking interface.

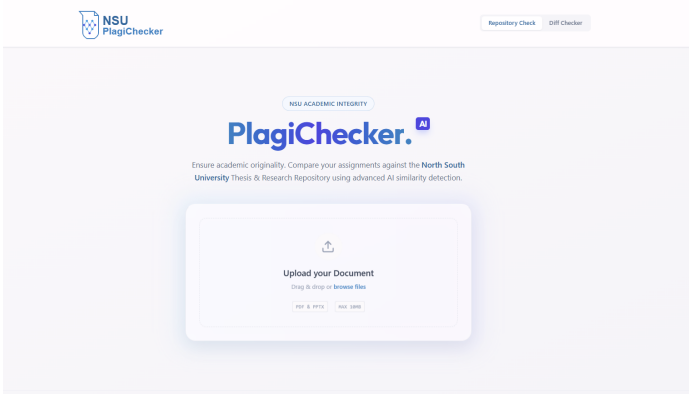

Fig. 2: Repository Check for large dataset plagiarism check

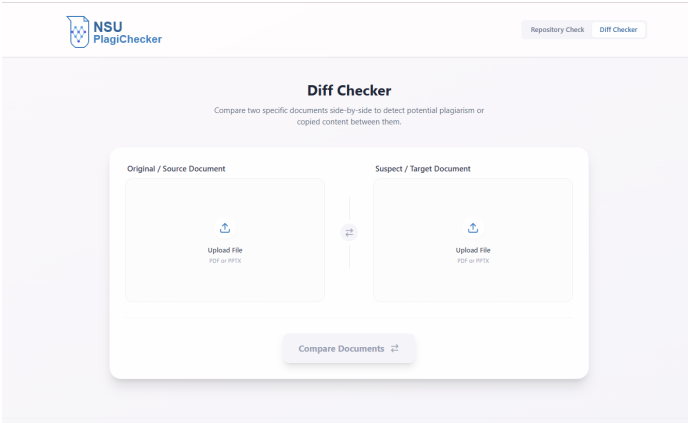

Fig. 3: Diff Checker for one-to-one PDF comparison

  - Inside the system, we planned two separate checking containers. The first one is the Repository Check, which is designed to compare one document against multiple reference files for large dataset plagiarism checking. The second one is the Diff Checker, which is designed for one-to-one PDF comparison.
  - Since this project is AI-based, we also started researching semantic similarity and vector embeddings to understand how plagiarism can be detected even when sentences are paraphrased.
  - Furthermore, we researched pretrained models under the Sentence-Transformers library to find a suitable model for similarity detection.
  - To maintain collaboration and version control, we created a GitHub repository and added group members as collaborators.
  - We also started designing the overall system architecture including repository indexing and document checking workflow.
- **Challenges faced:**
  - During the research phase, we faced several challenges while trying to understand how similarity detection actually works in plagiarism checking systems. One major difficulty was understanding how systems detect similarity when content is paraphrased. It was confusing to understand how AI models calculate similarity scores and how the percentage of similarity is determined between two texts.
  - Another challenge was researching how matched text can be highlighted inside PDF documents. Understanding how to locate the exact matching portion of text and visually highlight it inside the original document required additional research and experimentation.
  - We also faced difficulty while researching how to keep the system running locally instead of using cloud-based services. Since cloud solutions are commonly used for AI processing, finding suitable methods to implement document processing, vector storage, and similarity checking locally required extra study and testing.
  - To overcome these challenges, we followed online tutorials, technical documentation, and discussion forums which helped us understand these concepts gradually.
- **Progress made:**
  During this week, we successfully built the initial frontend layout including the landing page and submission interface. We also established our project collaboration environment through GitHub. In addition, we gained a basic understanding of semantic similarity techniques and finalized the initial architecture design of our plagiarism detection system. This gives us a strong foundation to start backend development in the upcoming weeks.

C. *Target for next weekly update*

- In the next week, we plan to start implementing the text processing pipeline of our system. Our main goal will be to extract text from uploaded PDF and PPTX files and prepare the text for similarity checking.
- We will research and finalize a method for PDF text extraction using libraries like PyPDF2 or pdfplumber. We plan to select one stable method and create a helper function that can extract text page by page.
- We will also implement PPTX text extraction using the python-pptx library so that slide contents can be collected and processed properly.
- After extracting the text, we will work on text cleaning techniques such as removing extra spaces, unwanted newline characters, and unnecessary special characters to make the text suitable for processing.
- We will then design and implement a chunking strategy where large documents will be divided into smaller segments based on token or word limits. We may also experiment with overlapping chunks to improve similarity detection accuracy.

- Additionally, we plan to design a metadata structure to store document information such as document ID, file name, file path, number of chunks, and indexing time. This will help us organize and manage repository documents efficiently.

II. Weekly Report 2

III. Weekly Report 3

IV. Weekly Report 4

V. Weekly Report 5